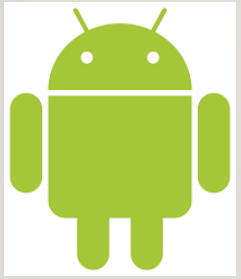
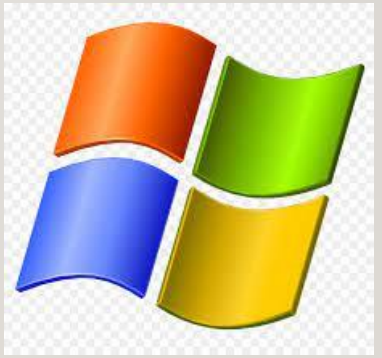
OPERATING SYSTEMS







COM 3306

H.M.C.J. DE SILVA

DEPARTMENT OF COMPUTING

SEMESTER PLAN

Week	Distribution	Evaluation
Week 1-Week 7	Launch Weeks	Quiz 01 – 10%
Week 7-Week 9	Mid Term Weeks	Quiz 02 – 10%
Week 9-Week 14	Momentum weeks	Assignment 01 – 10%

CONTENT FOR THE SEMESTER

- Introduction to OS
- Process Management
- Inter process communication and Synchronization
- Deadlock
- Memory management
- File Management
- Virtualization and cloud
- Security

CHAPTER I- INTRODUCTION TO OS

INTENDED LEARNING OUTCOMES (ILOS)

- Define what an operating system is and explain its importance.
- Identify the components of a computer system (hardware, software, users).
- Differentiate between system software and application software.
- Explain the main functions of an operating system.
- Describe different types of operating systems.
- Understand how users and programs interact with hardware through the OS.

WHY STUDY OPERATING SYSTEMS?

- OS is the backbone of modern computing
- Enables multitasking and multiuser systems
- Manages hardware efficiently
- Foundation for cybersecurity, cloud, virtualization

REAL LIFE EXAMPLES

- When multiple apps are open, OS manages CPU sharing.
- When RAM is full, OS uses virtual memory.
- When you save a file, OS decides how and where it is stored.
- When you use fingerprint/face unlock, OS manages security.

"If you want to control the computer efficiently, instead of just using it, you must understand Operating Systems."

COMPUTER SYSTEM

- Computer system is a collection of hardware and software components designed to provide an effective tool for computation.
- Hardware refers to electrical, mechanical, and electronic parts that make up the computer.
- Software refers to the set of programs written to provide services to the system. It gives life and meaning to hardware and brings it to the operational level.
- Software's are basically two types
    1. Application software
    2. System Software

COMPUTER SYSTEM

1. Application software
   - Set of programs written for a specific area of application.
     - Examples are word processors, spreadsheets, database management systems, Educational software, multimedia software
2. System software
   - Set of programs written for running computer hardware and applications and managing its resources such as memory, processors, and resources.
   - Provide a platform for running application software.
     - Operating systems, device drivers, antivirus programs

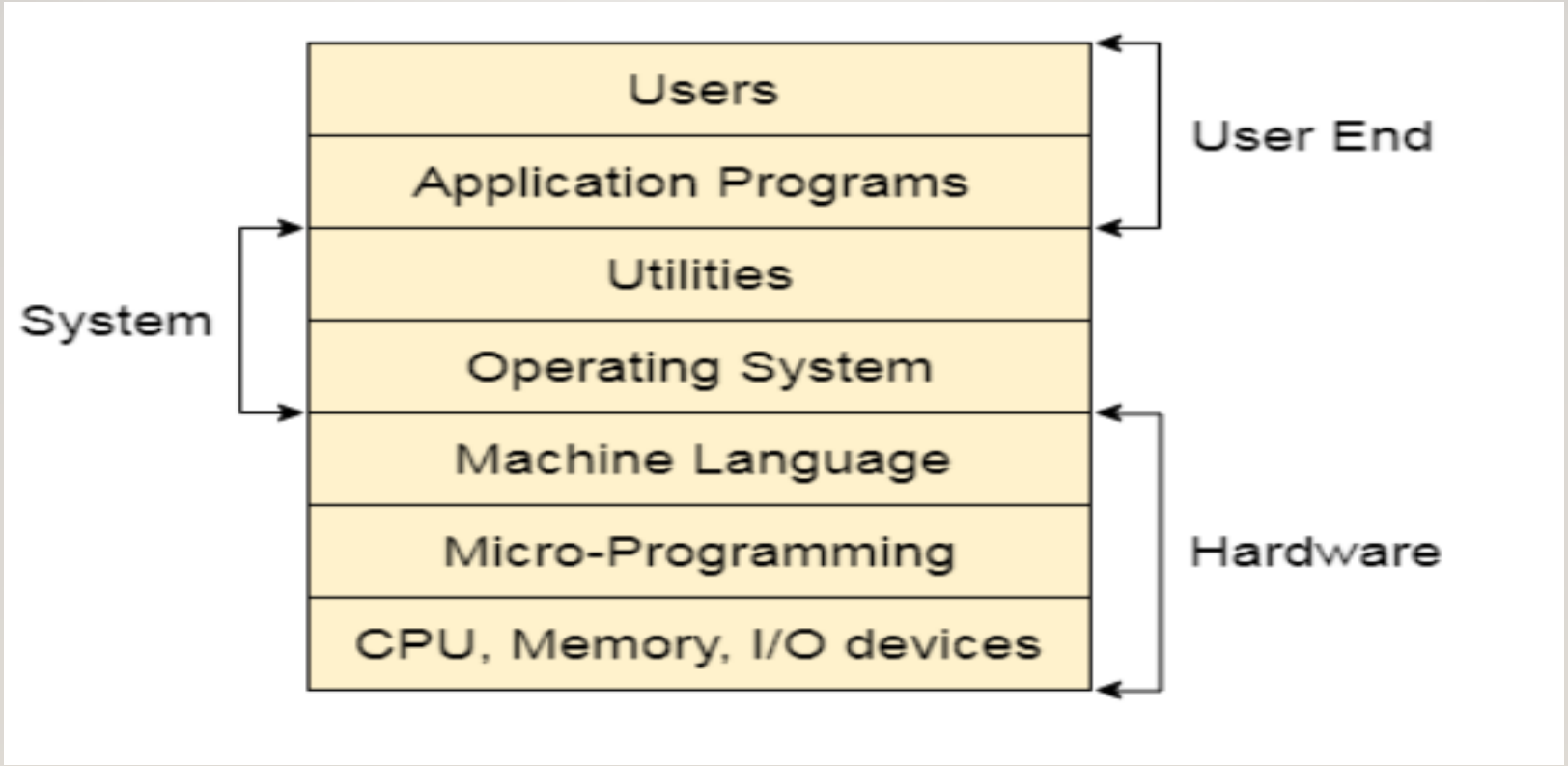
STRUCTURE OF A COMPUTER SYSTEM

- Computer system consists of:
  - **Users** -People who use the computer
  - **Application programs**-Compilers, Databases, Games, Video players, browsers, etc.
  - **System programs**-Shells, Editors, Compilers, etc.
  - **Operating system** -Special program which acts as a interface between the user and hardware
  - **Hardware**-CPU, Disk, memory, etc.

WHERE THE OS FITS IN A COMPUTER SYSTEM

- The Operating System (OS) is positioned **between the user/application programs and the computer hardware**.
- The OS acts as a **bridge/mediator** between software and hardware.

STRUCTURE OF A COMPUTER SYSTEM



TWO MODES OF OPERATION

- Computer system operate in two modes to ensure system protection:

1. User mode

- Used by application programs.
- Has limited access to hardware.
- Cannot execute privileged instructions.

2. Kernel mode/Supervisor mode

- Used by the Operating system
- Has full access to hardware and memory
- Can execute privileged instructions

WHAT IS AN OPERATING SYSTEM?

- An Operating System (OS) is system software that acts as an intermediary between the user and the computer hardware.
- An OS is a collection of system programs that together control the operation of a system.

  Examples-UNIX, MS-DOS, MS-Windows, Windows/NT, MacOS

- Operating systems indeed perform two fundamentally distinct yet equally important functions:

1. **The Operating System as an Extended Machine**
   - In this role, the operating system acts as a virtual machine, **abstracting away the complexities of the underlying hardware** and providing a simplified, easier-to-use interface to both programmers and users.
   - The messy, intricate details of hardware are hidden behind layers of abstraction, making it easier to interact with the system without needing to manage low-level operations. In essence, the OS is like a translator or intermediary that **makes the hardware user-friendly**

2. **The Operating System as a Resource Manager**

- While the OS simplifies the interface for users and applications, it simultaneously manages the **hardware resources(CPU, memory, devices)** of the computer to ensure that they are used efficiently and fairly.
- This role is crucial for preventing resource conflicts, ensuring that no one program hogs all the system resources, and enabling multiple processes to run concurrently.

HISTORY OF OPERATING SYSTEMS

1. Prehistory of operating systems(1940 to Early 1950)
   - No formal Operating systems
   - These early machines were entirely **manual systems**, and users needed to input data using **punch cards** or **plugboards**.
   - The programs had to be written specifically for each machine, and there was no concept of software layers or abstraction.
   - The **ENIAC** (Electronic Numerical Integrator and Computer) is an example of an early computer without an OS. Operators had to physically connect and set up the machine for each problem

HISTORY OF OPERATING SYSTEMS

2. First Generation of Operating Systems: Batch Systems (1950s to Early 1960s)

- Batch processing: Jobs were collected, or "batched," together and processed sequentially.
- The OS would load a job from a batch, execute it, and move on to the next without manual intervention.
- Early batch systems used a monitor program to manage the job queue.
- IBM 701 and IBM 7094 are examples

HISTORY OF OPERATING SYSTEMS

3. Second Generation of Operating Systems: Multiprogramming and Time-Sharing (1960s to 1970s)

- During the 1960s, computers became powerful enough to support multiple users simultaneously.
- This led to the development of multiprogramming and time-sharing systems, where multiple programs or users could run on the same machine by sharing system resources.
- Introduction of interrupts and job scheduling algorithms for better resource management.
- CTSS (Compatible Time-Sharing System) and MULTICS (Multiplexed Information and Computing Service)

HISTORY OF OPERATING SYSTEMS

4. Third Generation General-Purpose Operating Systems (1970s to 1980s)
   - The birth of operating systems that supported minicomputers and personal computers.
   - General-purpose systems: Operating systems were developed to run on different hardware platforms, breaking away from single-purpose systems.
   - Support for virtual memory, which allowed programs to use more memory than was physically available by paging parts of programs in and out of physical memory.
   - Increased emphasis on portability: OS could be moved to different hardware architectures with minimal modifications.
   - UNIX designed as a portable, multi-tasking, multi-user operating system

HISTORY OF OPERATING SYSTEMS

5. Fourth Generation : PC Operating Systems and GUIs (1980s to 1990s)

- Personal computers: Operating systems were developed specifically for desktop machines used by individuals.
- Introduction of GUIs: GUIs replaced command-line interfaces (CLIs) to make systems more user-friendly.
- Integration of networking capabilities, leading to the early stages of networked computing and the rise of the internet.
- MS-DOS, Windows 1.0 and Apple macintosh

HISTORY OF OPERATING SYSTEMS

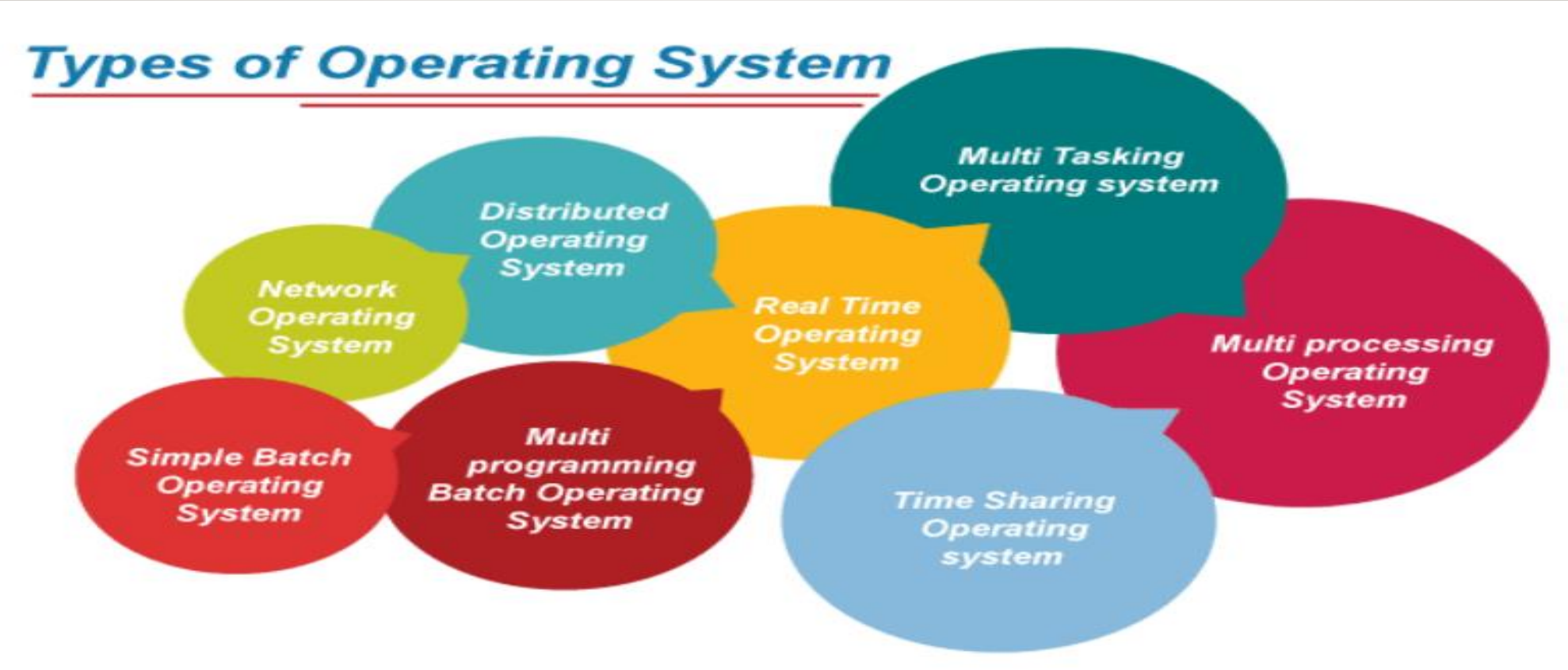
6. Fifth Generation : Modern Operating Systems(1990s to present)
   - Operating systems evolved rapidly to accommodate advancements in hardware, networking, and user demands.
   - The growth of the internet, mobile computing, and the rise of distributed systems and virtualization have all shaped modern operating systems
   - Windows NT,Linux,macOS,Android and iOS

HISTORY OF OPERATING SYSTEMS

7. Sixth Generation : Cloud, Virtualization, and Distributed Systems (2000s to Present)

- Cloud computing: Many applications now run on remote servers (in the cloud), with operating systems managing large, distributed infrastructures. Examples include Amazon Web Services (AWS) and Google Cloud.
- Virtualization: Technologies like VMware, KVM, and Hyper-V allow multiple operating systems to run on the same physical hardware, facilitating cloud services, development, and testing.
- Containerization: Systems like Docker and Kubernetes allow applications to be deployed in isolated containers that share the OS kernel but run independently

TYPES OF OPERATING SYSTEMS



TYPES OF OPERATING SYSTEMS

- **Batch Operating System**

-Access is given to more than one person; they submit their respective jobs to the system for execution.

-The system put all of the jobs in a queue on the basis of first come first serve and executes jobs one by one.

-Users get their respective outputs when all the jobs get executed.

-Purpose of this OS was to transfer the control from one job to another as soon as the job was completed.

TYPES OF OPERATING SYSTEMS

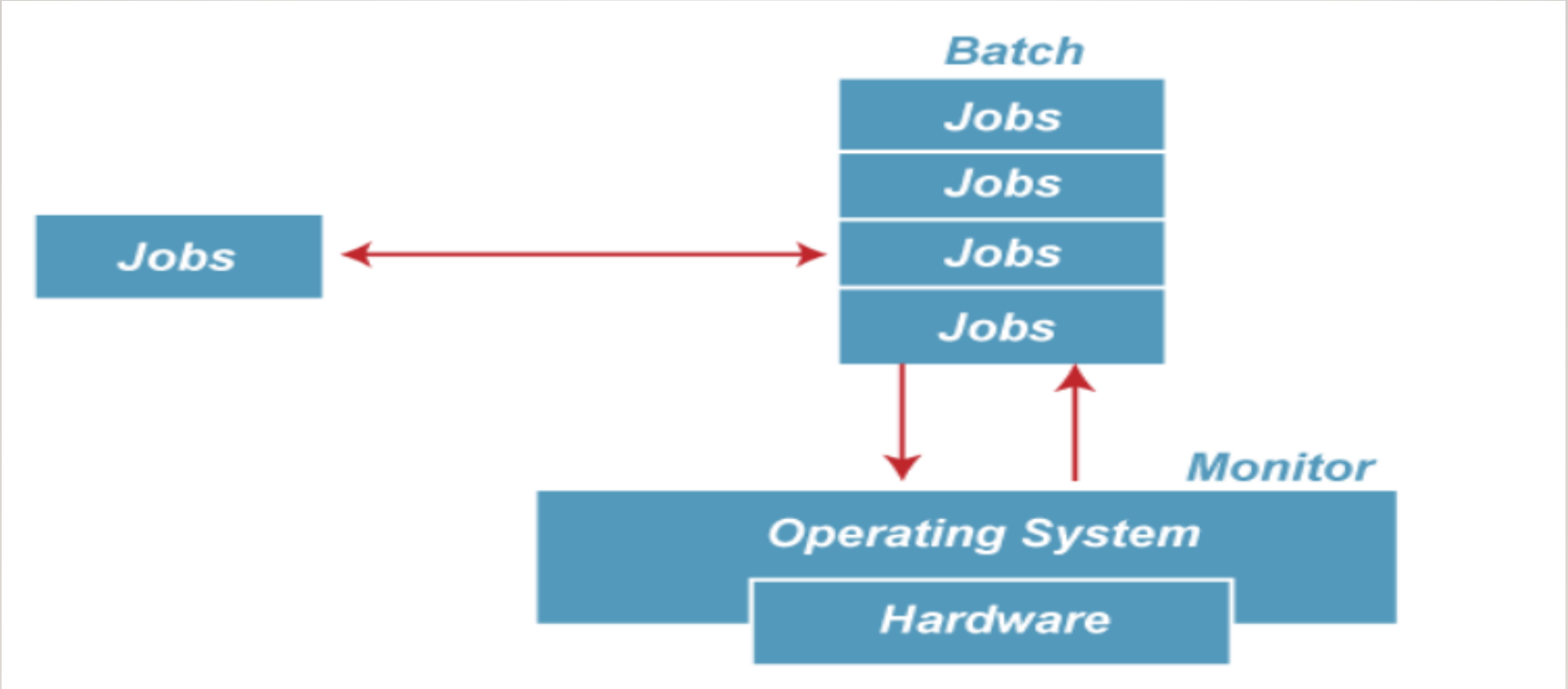
Advantages

- The use of a resident monitor improves computer efficiency as it eliminates CPU time between two jobs.

Disadvantages

- Starvation
- Not interactive

TYPES OF OPERATING SYSTEMS



TYPES OF OPERATING SYSTEMS

- **Multiprogramming OS**

-Extension to batch processing where the CPU is always kept busy.

-Each process needs two types of system time: CPU time and IO time.

-In a multiprogramming environment, when a process does its I/O, the CPU can start the execution of other processes.

Advantages

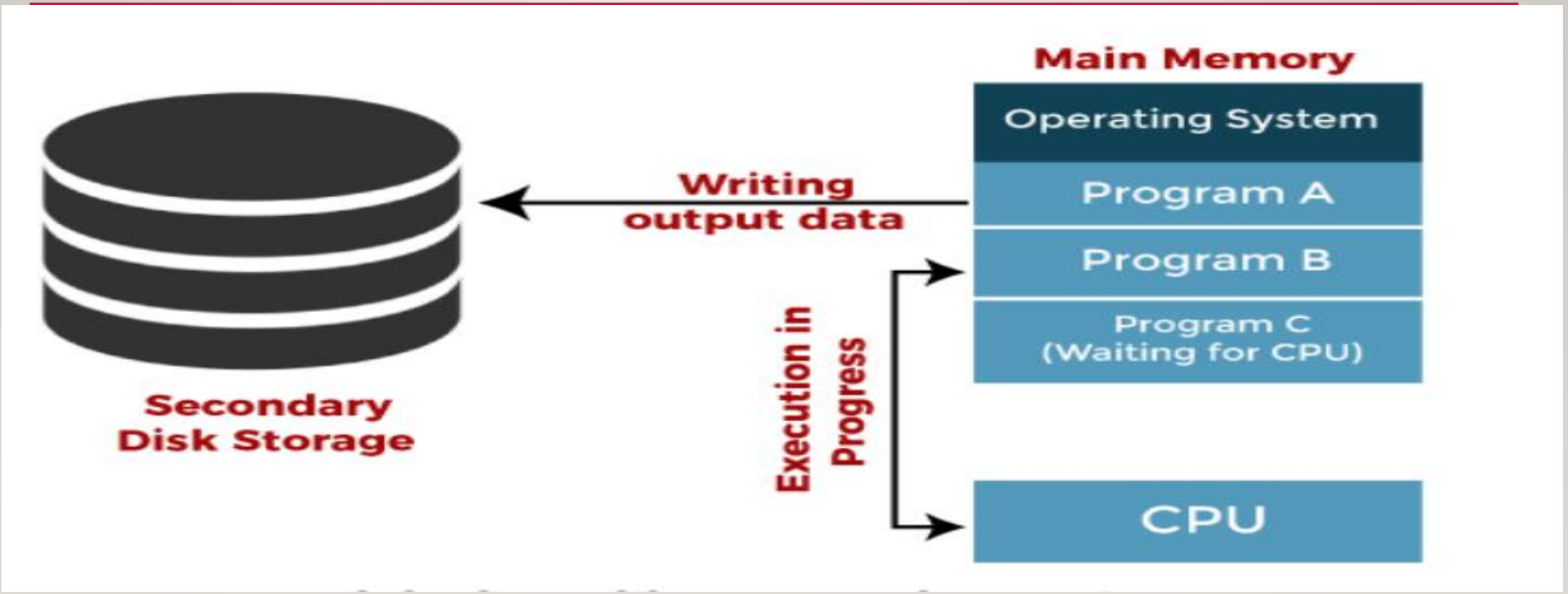
- Throughput of the system is increased as the CPU always had one program to execute.
- Response time can also be reduced.

TYPES OF OPERATING SYSTEMS

Disadvantages

- Various systems resources are used efficiently, but they do not provide any user interaction with the computer system.

TYPES OF OPERATING SYSTEMS



TYPES OF OPERATING SYSTEMS

- **Multiprocessing OS**

-Parallel computing is achieved by multiprocessing.

-There is more than one processor present in the system which can execute more than one process at a time.

-This increases the throughput of the system.

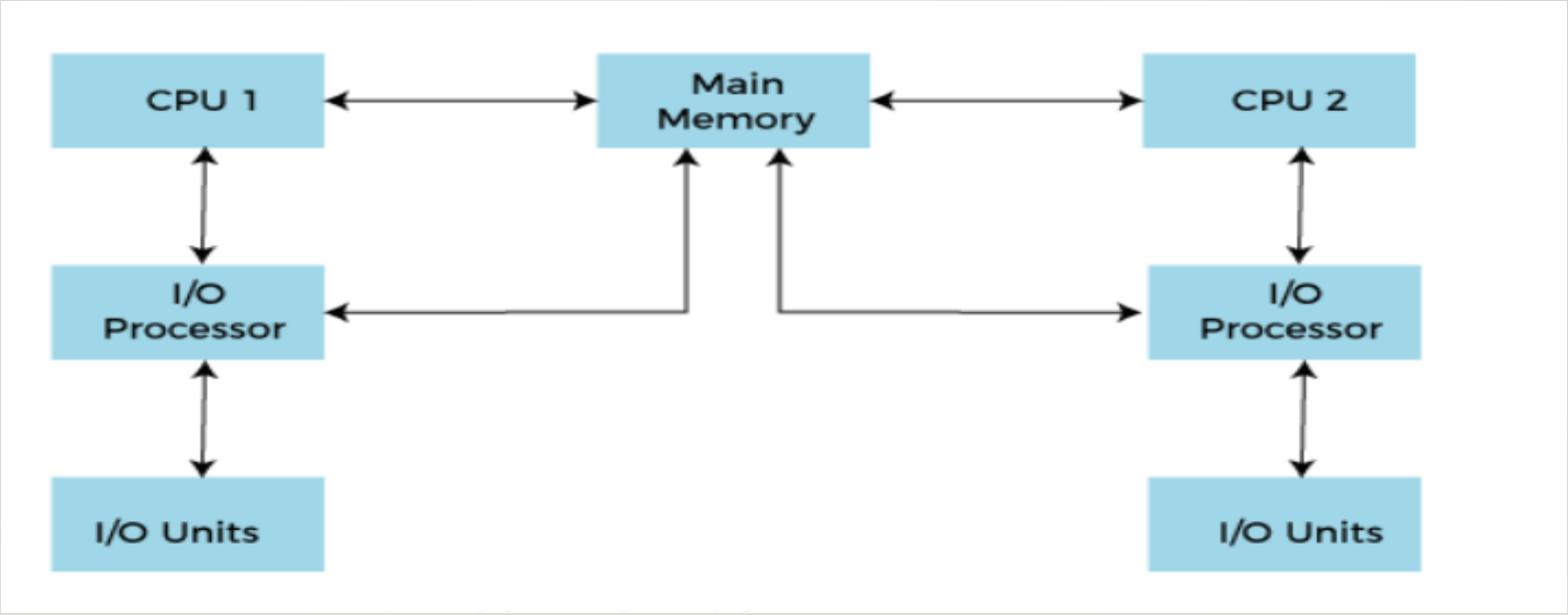
Advantages

- Increased reliability
- Increased throughput

TYPES OF OPERATING SYSTEMS

Disadvantages

- Complex and sophisticated as it takes care of multiple CPUs simultaneously.



TYPES OF OPERATING SYSTEMS

- **Multitasking OS**

-Logical extension of multiprogramming system that enables multiple programs simultaneously.

-It allows a user to perform more than one computer task at the same time.

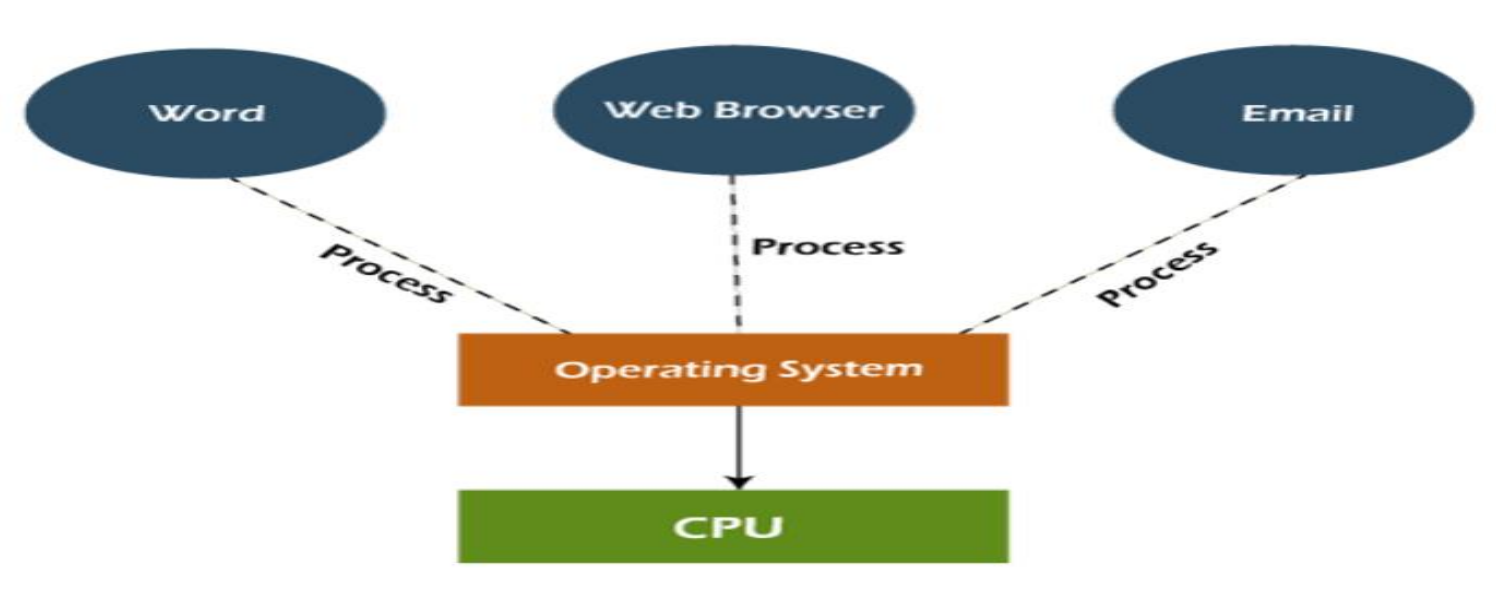
Advantages

- More suited to support multiple users simultaneously.
- Have well defined memory management.

TYPES OF OPERATING SYSTEMS

Disadvantage

- Busier at the same time to complete any task in a multitasking environment, so the CPU generates more heat.



TYPES OF OPERATING SYSTEMS

- **Network Operating System**

-Include software and associated protocols to communicate with other computers via the network conveniently and cost-effectively.

-Types of network OS:

1.Peer to peer

2.Client-server system

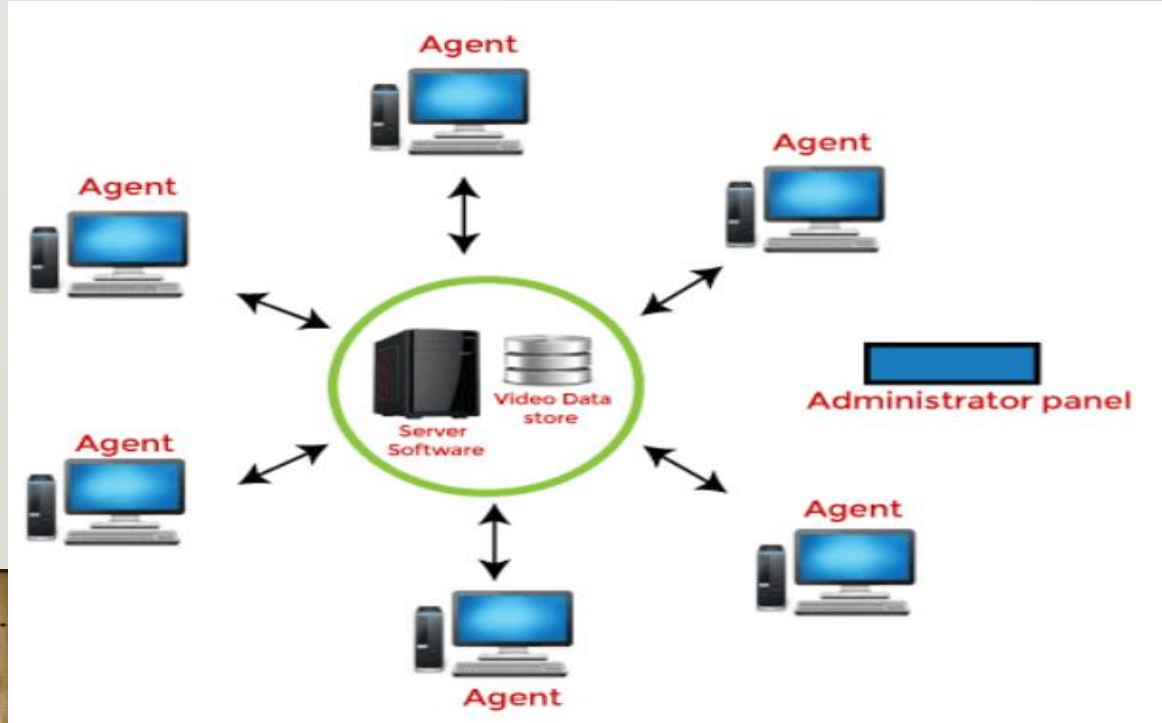
Advantages

- network traffic reduces due to the division between clients and the server.
- This type of system is less expensive to set up and maintain.

TYPES OF OPERATING SYSTEMS

Disadvantages

- Failure of any node in the system affects the whole system.
- Trained network administrators are required.



TYPES OF OPERATING SYSTEMS

- **Real Time Operating System**

-Each job carries a certain deadline within which the job is supposed to be completed, otherwise huge loss will be there or even if the result is produced it will be completely useless.

-The Application of a Real-Time system exists in the case of military applications, if you want to drop a missile, then the missile is supposed to be dropped with a certain precision.

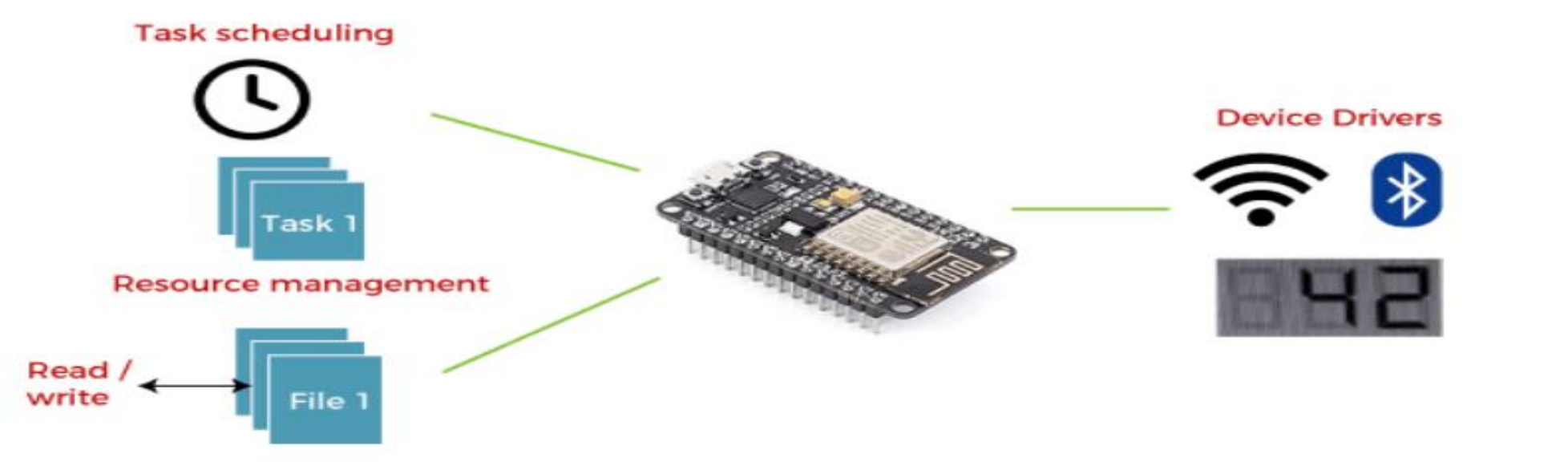
Advantages

- Easy to layout, develop, and execute real-time applications under real-time OS
- Maximum utilization of devices and systems.

TYPES OF OPERATING SYSTEMS

Disadvantages

- Very costly and develop
- Very complex and consume critical CPU cycles.



TYPES OF OPERATING SYSTEMS

- **Time-sharing OS**

-Computer resources are allocated in a time-dependent fashion to several programs simultaneously.

-Thus it helps to provide a large number of user's direct access to the main computer

-It is a logical extension of multiprogramming.

-In time-sharing, the CPU is switched among multiple programs given by different users on a scheduled basis.

TYPES OF OPERATING SYSTEMS

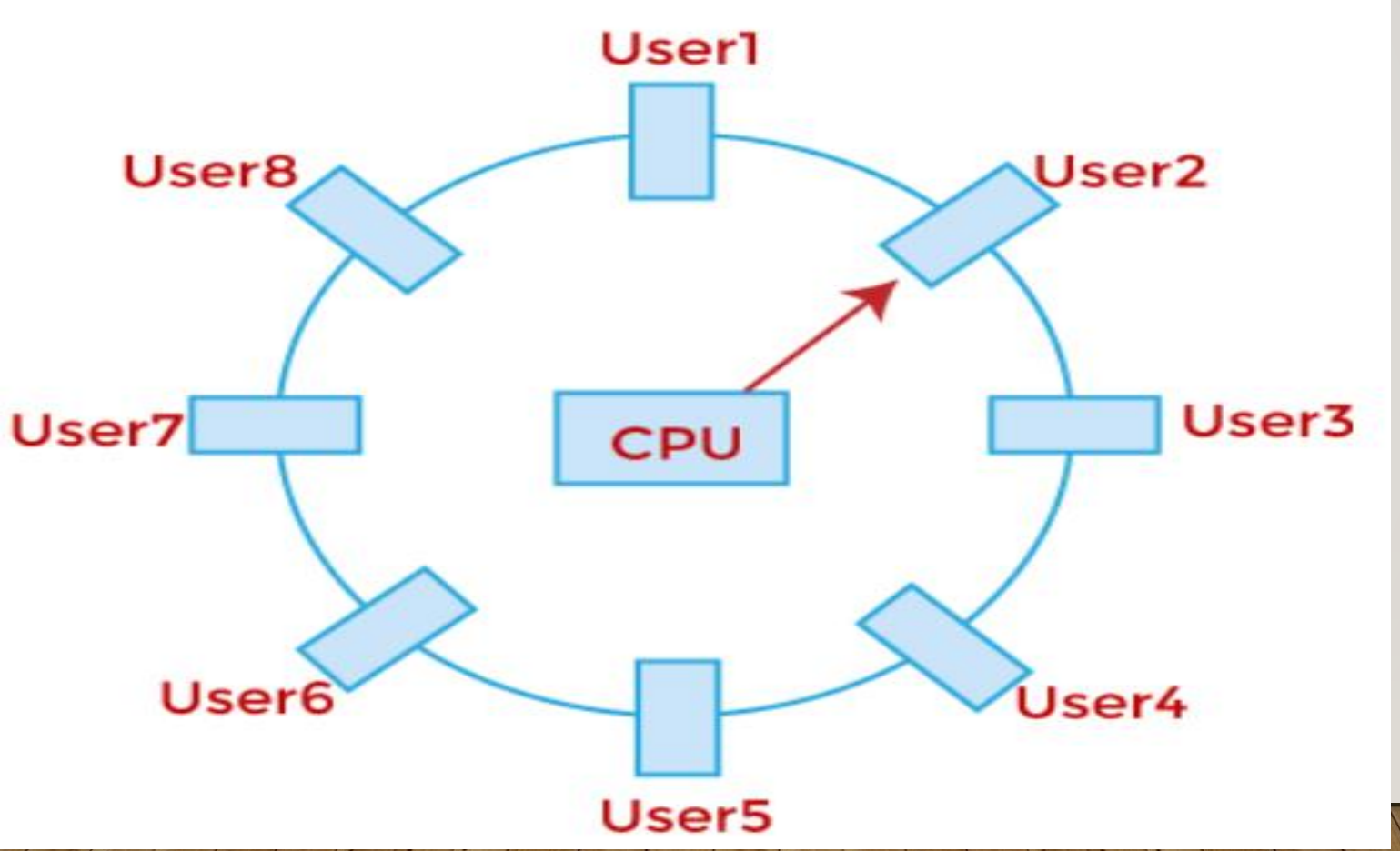
Advantages

- Provide effective utilization and sharing of resources
- System reduces CPU idle and response time

Disadvantages

- Data transmission rates are very high compared to other methods.
- Security and integrity of user programs loaded in memory and data need to be maintained as many users access the system at the same time.

TYPES OF OPERATING SYSTEMS



TYPES OF OPERATING SYSTEMS

- **Distributed OS**

-Not installed in a single machine, distributed into parts and these parts are loaded on different machines.

- A part of the distributed Operating system is installed on each machine to make their communication possible.

-More complex, large, and sophisticated than Network operating systems because they also have to take care of varying networking protocols.

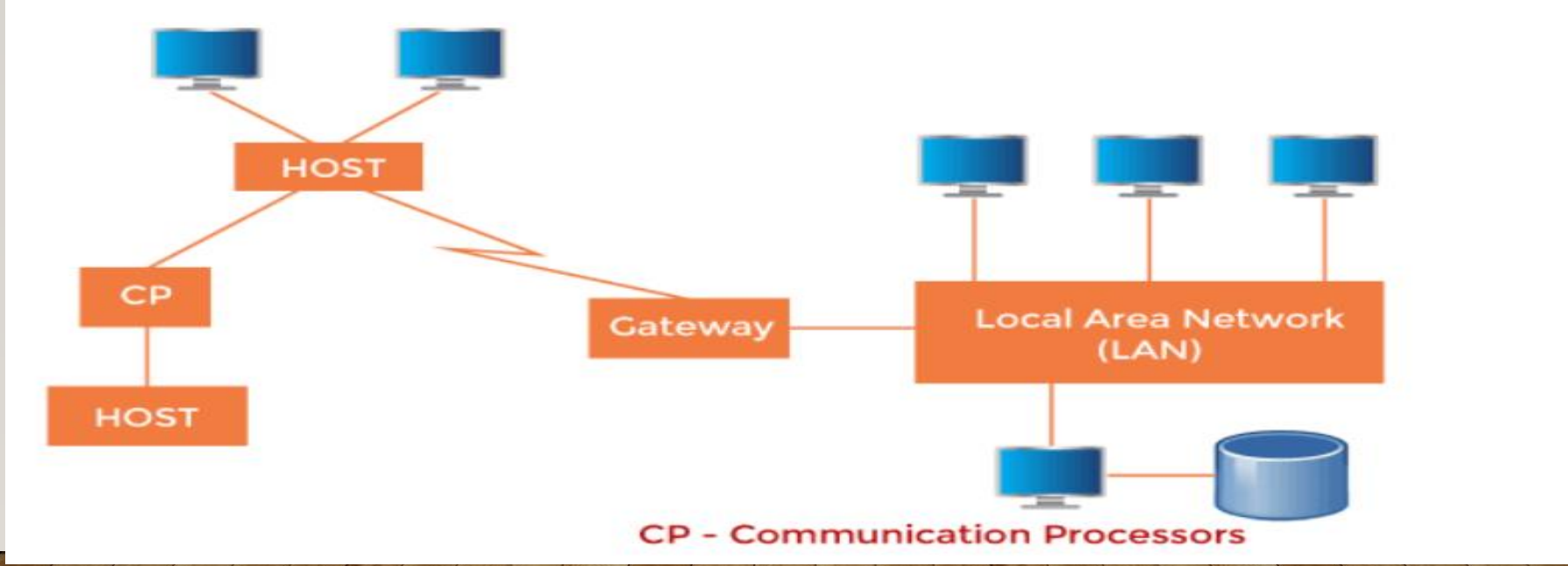
Advantages

- Provide sharing resources
- Fault tolerate

TYPES OF OPERATING SYSTEMS

Disadvantages

- Protocol overhead can dominate computation cost.



OPERATING SYSTEM CONCEPTS

- **Processes**
  - Program in execution
  - Each process has its program code, data section, stack, CPU registers and process ID.
  - Each process uses CPU, memory and I/O resources.
  - The address space contains the executable program, the program's data, and its stack.
  - Example- Running a text editor is a process, and if you open two instances of the text editor, each instance is a separate process.

OPERATING SYSTEM CONCEPTS

- **Memory space**
- Physical memory (RAM) used by the system.
- Shared among all the processes and this is managed by OS.

OPERATING SYSTEM CONCEPTS

- **Address Space**
  - Every computer has some main memory that it uses to hold executing programs.
  - In a very simple operating system, only one program at a time is in memory. To run a second program, the first one has to be removed, and the second one placed in memory.
  - More sophisticated operating systems allow multiple programs to be in memory at the same time.
  - Address space is logical view of memory for a process.
  - This is private to each process, and this provides isolation and security prevent memory corruptions and enable multi tasking.

OPERATING SYSTEM CONCEPTS

- **Input/Output**
  - All computers have physical devices for acquiring input and producing output.
  - Consequently, every operating system has an I/O subsystem for managing its I/O devices.
  - Some of the I/O software is device independent, that is, applies to many or all I/O devices equally well. Other parts of it, such as device drivers, are specific to particular I/O devices.
  - Handles input and output devices.
  - Uses device drivers.

OPERATING SYSTEM CONCEPTS

- **Files**
- Permanent data storage
- Stored on secondary memory
- Managed by OS

OPERATING SYSTEM CONCEPTS

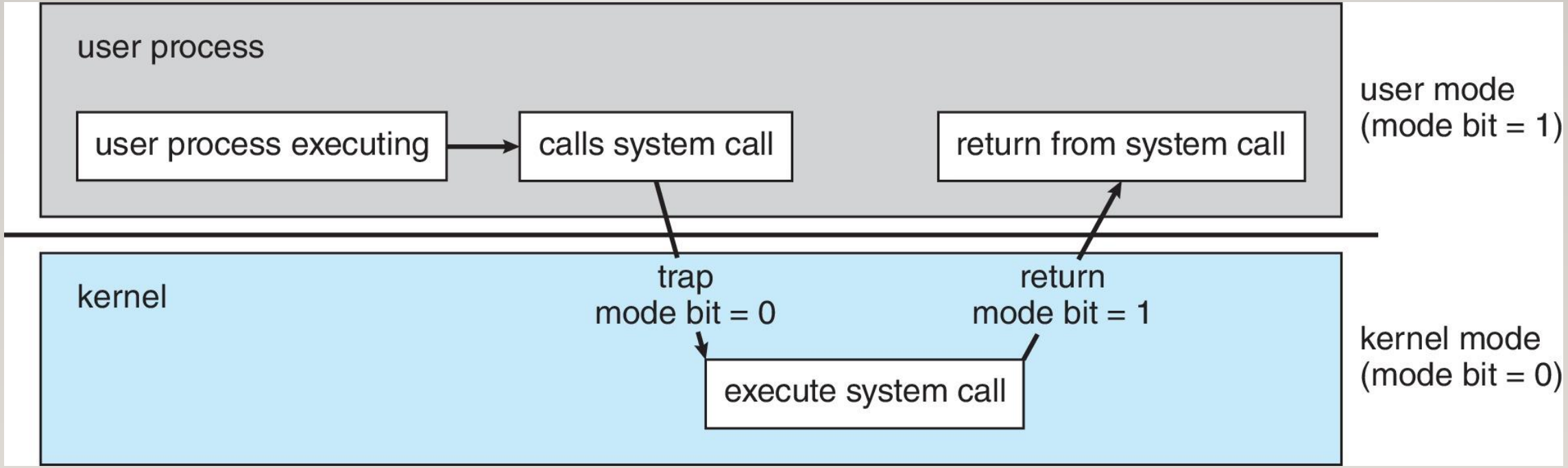
- **The shell**
  - The shell is an interface that allows users to interact with the operating system, typically via command-line input. The shell acts as an intermediary between the user and the OS's kernel.
  - The shell interprets user commands (such as creating files, listing directory contents, or launching programs) and passes them to the kernel for execution.
  - Types of Shells:
    - Command-Line Interface (CLI): Text-based interface where users type commands. Examples: Bash (Linux), Command Prompt (Windows), Zsh (macOS).
    - Graphical User Interface (GUI): A more user-friendly, graphical interface where users interact with the system through icons and windows.Examples: Windows Explorer, macOS Finder.

OPERATING SYSTEM CONCEPTS

- **System calls**
  - System call is a mechanism that allows a user program to request services from the operating system.
  - User programs cannot directly access hardware or critical system resources, so they must use system calls to perform such operations.
  - System calls act as an interface between user mode and kernel mode.
  - When a system call is invoked, the CPU switches from user mode to kernel mode to safely execute the requested operation.
  - After the operation is completed, control is returned to the user program and the CPU switches back to user mode.

OPERATING SYSTEM CONCEPTS

- **System Calls**



OPERATING SYSTEM CONCEPTS

- **System calls**
  - System calls are necessary to ensure system security, stability, and controlled access to hardware resources.
    They prevent user programs from directly manipulating memory, CPU, or I/O devices.
    Common services provided through system calls include process creation, file handling, memory management, and input/output operations.

FUNCTIONS OF OPERATING SYSTEMS

1. **Process Management**

- A process is a program in execution.
- Process needs specific resources, including CPU time, files, and I/O devices to accomplish its task.
- OS is responsible for following activities in connection with process management.
  - Create, load, execute, suspend, resume, and terminate Processes.
  - Switch system among multiple processes in main memory.
  - Provide communication mechanisms so that processes can communicate with each others.

FUNCTIONS OF OPERATING SYSTEMS

- Provide synchronization mechanisms to control concurrent access to shared data to keep shared data consistent.
- Allocate/de-allocate resources properly to prevent or avoid deadlock situations.

2. **File management**

- File is a collection of related information defined by its creator.
- Most files represent data, source, and object forms and programs.
- Data files can be of any type like alphabetic, numeric, and alphanumeric.
- Also files are normally organized into directories to ease their use and these directories can contain files and other directories.

FUNCTIONS OF OPERATING SYSTEMS

- OS is responsible for following activities in connection with file management.
  - File creation and deletion.
  - Directory creation and deletion
  - The support of primitives for manipulating files and directories.
  - Mapping files onto secondary storage.

FUNCTIONS OF OPERATING SYSTEMS

3. I/O Device management

- Hide the peculiarities of specific hardware devices from the user.
- I/O Device Management provides an abstract level of H/W devices and keeps the details from applications to ensure proper use of devices, prevent errors, and provide users with a convenient and efficient programming environment.
- Taks of I/O device management components are,
  - Hide the details of Hardware devices.
  - Manage main memory for the devices using cache, buffer, and spooling.
  - Maintain and provide custom drivers for each device,

FUNCTIONS OF OPERATING SYSTEMS

4. Network management

- Network management is the process of keeping your network healthy for efficient communication between different computers.
- Computer network is a collection of various types of computers connected with each other.
- Network management comprises fault analysis, maintaining the quality of service, provisioning of networks, and performance management.
- Featured of network management,
  - Network administration
  - Network maintenance

FUNCTIONS OF OPERATING SYSTEMS

- Network operation
- Network provisioning
- Network security

5. Main memory management

- Main memory is a repository of quickly accessible data shared by the CPU and I/O devices.
- Main memory is a volatile storage device.
- The main motivation behind Memory Management is to maximize memory utilization on the computer system.

FUNCTIONS OF OPERATING SYSTEMS

- OS is responsible for following activities in connection with memory management.
  - Keep tracks of which parts of memory are currently being used and by whom.
  - Decide which processes to load when memory space becomes available.
  - Allocate and deallocate memory space as needed.

6. Secondary Storage Management

- Programs and the data they access must be in the main memory during execution to execute programs.
- Since the main memory is too small to permanently accommodate all data and programs, the computer system should provide secondary storage to back up the main memory.

FUNCTIONS OF OPERATING SYSTEMS

- Most modern computer systems use disks as the principal online storage medium for both programs and data.
- Most programs, like compilers, assemblers, sort routines, editors, formatters, and so on, are stored on the disk until loaded into memory and then use the disk as both the source and destination of their processing.
- OS is responsible for following activities,
  - Free Space management
  - Storage allocation

FUNCTIONS OF OPERATING SYSTEMS

7. Security Management

- Security Management refers to a mechanism for controlling the access of programs, processes, or users to the resources defined by computer controls to be imposed, together with some means of enforcement.
- For example memory addressing hardware ensures that the process can only execute within its own address space.
- Timer ensures that no process can gain control of the CPU without turning over it.

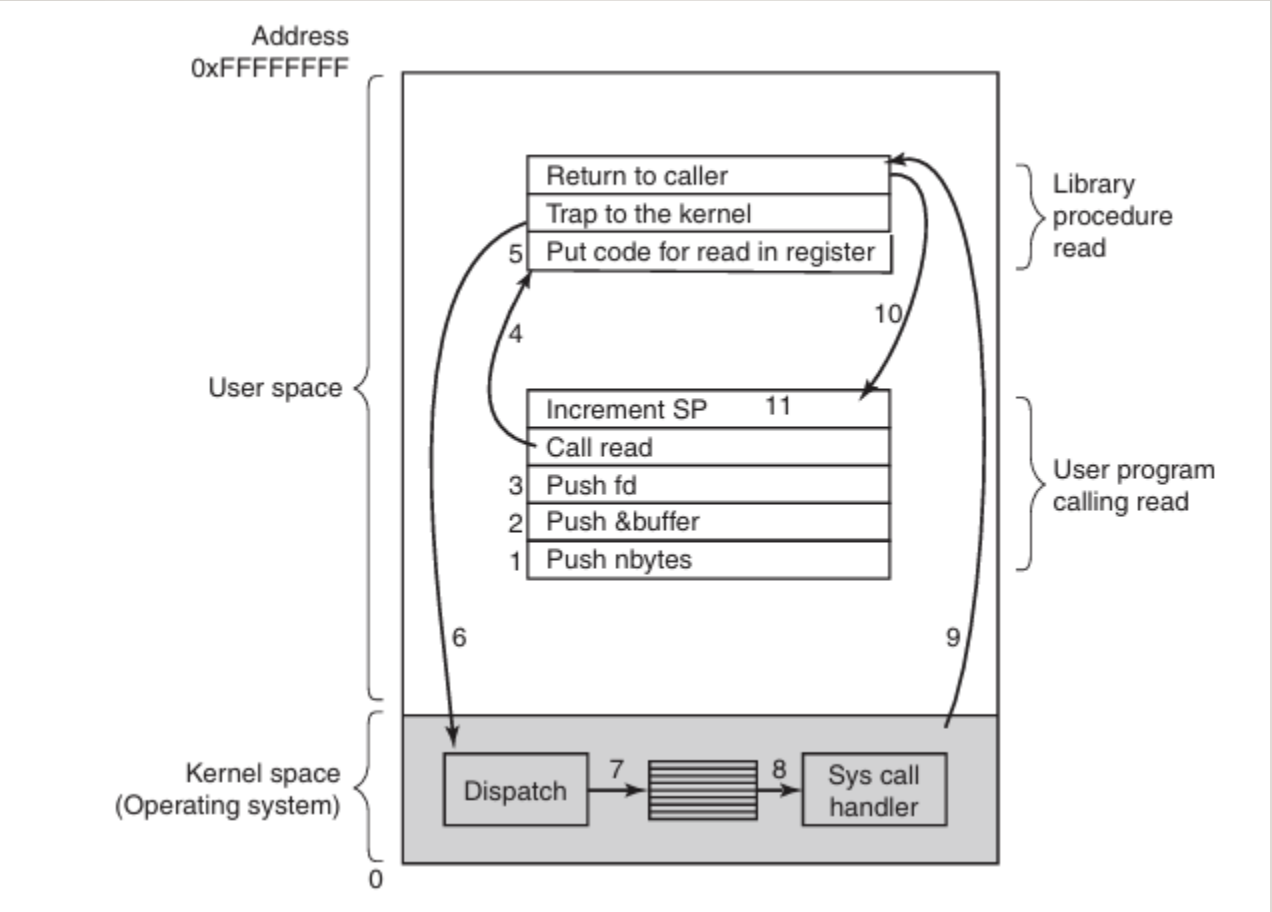
GOALS OF OS

- **Convenience** -An OS makes a computer more convenient for the user for using.
- **Efficiency** -An OS allows the computer system resources to be used in an efficient manner, to ensure good resource utilization efficiency and provide appropriate corrective actions when it becomes low
- **Ability to evolve** -An OS should construct in such a way as to permit the effective development, testing, and introduction of new system functions without interfering with service.

SYSTEM CALLS

- Operating systems perform two main functions: providing abstractions for user programs (like file management) and managing computer resources.
- The interaction between user programs and the OS primarily involves the abstractions, with system calls being the interface.
- System calls differ across operating systems, but the concepts are similar.
- To illustrate, a specific example is the POSIX read system call, which has three parameters: the file descriptor, a buffer for data, and the number of bytes to read.

- System calls are made by user programs via procedure libraries, like read().
- When the system call is executed, it switches the process from user mode to kernel mode using a TRAP instruction, which also saves the return address.
- The kernel inspects the system call number and dispatches the request to the correct handler.
- Once the work is done, control is returned to the user program.
- This interaction between user programs and the OS reveals how system calls act like special procedure calls, except they enter the kernel, manage hardware and resources, and handle operations like I/O, file management, and process control.
-



- The 11 steps in making the system call read(fd, buffer, nbytes)

Step 1
The user program pushes the number of bytes to be read (`nbytes`) onto the stack.
Step 2
The user program pushes the address of the buffer (`&buffer`) onto the stack, where the read data should be stored.
Step 3
The user program pushes the file descriptor (`fd`) onto the stack, which identifies the open file.
Step 4
The user program calls the library procedure `read()`.
Step 5
The library procedure places the system call number for `read` into a CPU register.
Step 6
The library procedure executes a **trap instruction**, which switches the CPU from **user mode to kernel mode** and transfers control to the operating system.
Step 7
The operating system's **dispatcher** examines the system call number to determine which system call has been requested.
Step 8
The dispatcher transfers control to the appropriate **system call handler** for `read()`.

Step 9
The system call handler performs the requested operation by reading data from the file into the buffer and then returns control to user mode.
Step 10
Control returns from the kernel to the library procedure, which then prepares to return to the user program.
Step 11
The library procedure increments the stack pointer to remove the parameters from the stack and returns control to the calling user program.

System calls for Process Management

- System calls in this category allow the operating system to manage processes. They enable the creation, termination, and control of processes.
- These system calls are critical for process creation, execution, synchronization, and termination in multitasking environments.
  - fork(): Creates a new process by duplicating the calling process.
  - exec(): Replaces the current process memory with a new program.
  - exit(): Terminates the calling process and returns a status code.
  - wait(): Waits for a child process to finish execution.
  - getpid(): Retrieves the process ID of the calling process.
  - kill(): Sends a signal to a process, which may cause it to terminate.

System calls for file management

- These system calls allow user programs to interact with files. Operations like creating, reading, writing, and closing files fall under this category.
- File management system calls are fundamental to performing I/O operations and manipulating file data.
  - open(): Opens a file and returns a file descriptor.
  - read(): Reads data from a file into a buffer.
  - write(): Writes data from a buffer to a file.
  - close(): Closes a file descriptor.
  - lseek(): Repositions the file pointer within a file.
  - unlink(): Deletes a file.

System calls for directory management

- These system calls allow interaction with directories, including creating, deleting, and reading directory contents.
- Directory management is essential for organizing files into a hierarchical structure.
  - mkdir(): Creates a new directory
  - mdir(): Removes an empty directory.
  - opendir(): Opens a directory stream.
  - readdir(): Reads the contents of a directory.
  - closedir(): Closes an open directory stream.
  - rename(): Renames a file or directory.

Miscellaneous system calls

- These calls provide additional services, including system information and time management.
- These miscellaneous system calls handle tasks related to time, user identity, and system information.
  - gettimeofday(): Returns the current time.
  - sleep(): Suspends execution for a specified duration.
  - alarm(): Sets a timer to send a signal after a specified number of seconds.
  - gethostname(): Retrieves the name of the machine on the network.
  - getuid(): Gets the user ID of the calling process

The windows Win32 API

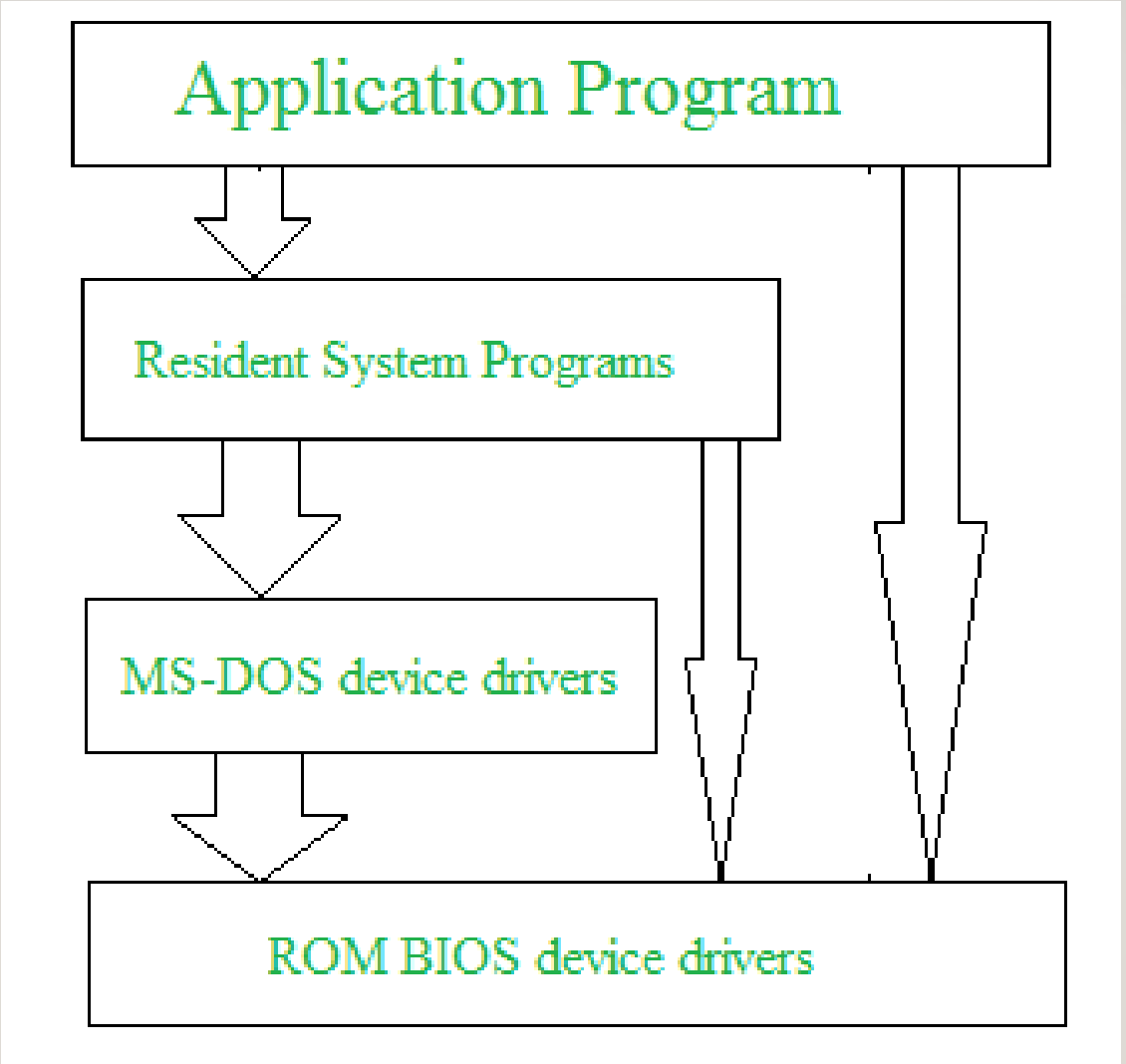
- In Windows, the **Win32 API** is the primary interface for system calls.
- It provides functions for process management, file operations, I/O handling, and much more. Some of the key functions include:
  - Process Management: CreateProcess(), TerminateProcess(), WaitForSingleObject()
  - File Management: CreateFile(), ReadFile(), WriteFile(), CloseHandle()
  - Thread Management: CreateThread(), ExitThread(), WaitForMultipleObjects()
  - Memory Management: VirtualAlloc(), VirtualFree()
  - I/O Operations: SetFilePointer(), GetFileSize()

OPERATING SYSTEM STRUCTURE

- An operating system (OS) is a complex software that manages hardware resources and provides services to applications.
- To handle these tasks efficiently, operating systems are structured in specific ways.
- There are several common approaches to structuring an operating system.
  - Simple/Monolithic Structure
  - Micro-Kernel Structure
  - Hybrid-Kernel Structure
  - Exo-Kernel Structure
  - Layered Structure
  - Modular Structure
  - Virtual Machines

Simple/Monolithic structure

- No well-defined structure and are small,simple and limited.
- The interfaces and levels of functionality are not well separated.
- MS-DOS is an example of such an operating system. In MS-DOS, application programs are able to access the basic I/O routines.
- These types of operating systems cause the entire system to crash if one of the user programs fails
- **Advantages**: Fast execution due to direct interaction among system components.
- **Disadvantages**: The structure is very complicated, as no clear boundaries exist between modules.


Application Program
Resident System Programs
MS-DOS device drivers
ROM BIOS device drivers

Micro kernel structure

- This structure designs the operating system by removing all non-essential components from the kernel and implementing them as system and user programs.
- This results in a smaller kernel called the micro-kernel.
- Advantages of this structure are that all new services need to be added to user space and does not require the kernel to be modified.
- Thus it is more secure and reliable as if a service fails, then rest of the operating system remains untouched. Mac OS is an example of this type of OS.

Advantages of Micro-kernel Structure

- It makes the operating system portable to various platforms.
- As microkernels are small so these can be tested effectively.

Disadvantages of Micro-kernel Structure

- Increased level of inter module communication degrades system performance.

Hybrid kernel structure

- Hybrid-kernel structure is nothing but just a combination of both monolithic-kernel structure and micro-kernel structure.
- Basically, it combines properties of both monolithic and micro-kernel and make a more advance and helpful approach.
- It implement speed and design of monolithic and modularity and stability of micro-kernel structure.
- Advantages:
  - It offers good performance as it implements the advantages of both structure in it.
  - It supports a wide range of hardware and applications.
  - It provides better isolation and security by implementing micro-kernel approach.

Hybrid kernel structure

- Disadvantages:
  - It increases overall complexity of system by implementing both structure (monolithic and micro) and making the system difficult to understand.
  - The layer of communication between micro-kernel and other component increases time complexity and decreases performance compared to monolithic kernel.

Exo-kernel structure

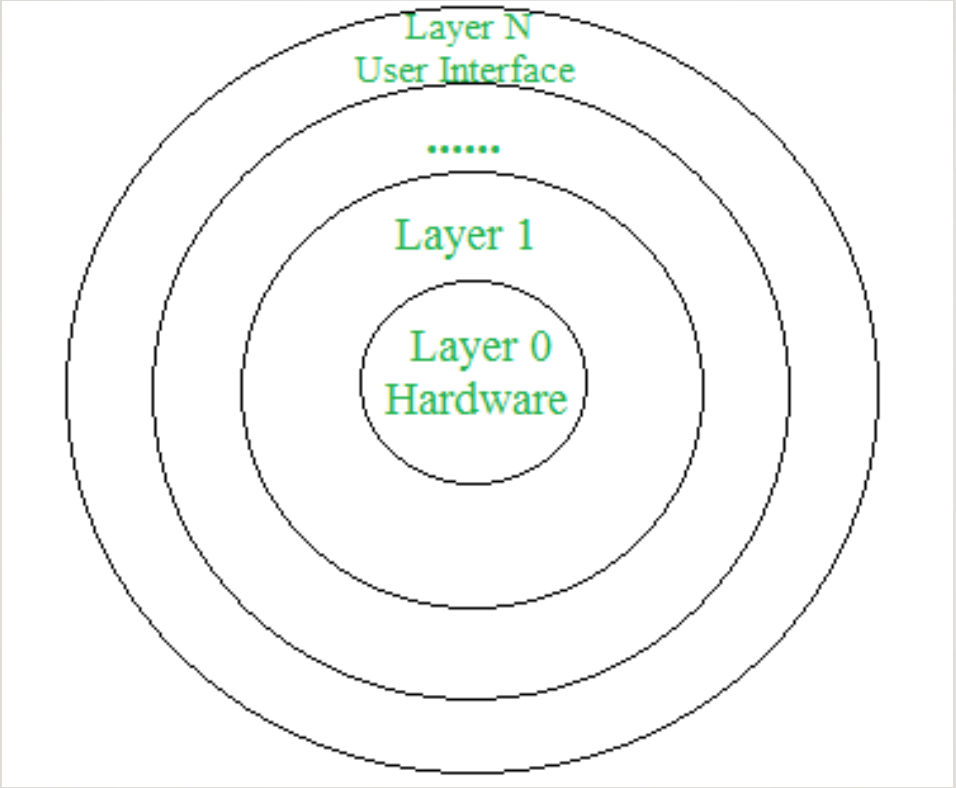
- Exokernel is an operating system developed at MIT to provide application-level management of hardware resources.
- By separating resource management from protection, the exokernel architecture aims to enable application-specific customization.
- Due to its limited operability, exokernel size typically tends to be minimal.
- Advantages:
  - Support for improved application control.
  - Separates management from security.
  - It improves the performance of the application.
  - A more efficient use of hardware resources is made possible by accurate resource allocation and revocation.

Exo-kernel structure

- Disadvantages:
  - A decline in consistency.
  - Exokernel interfaces have a complex architecture.

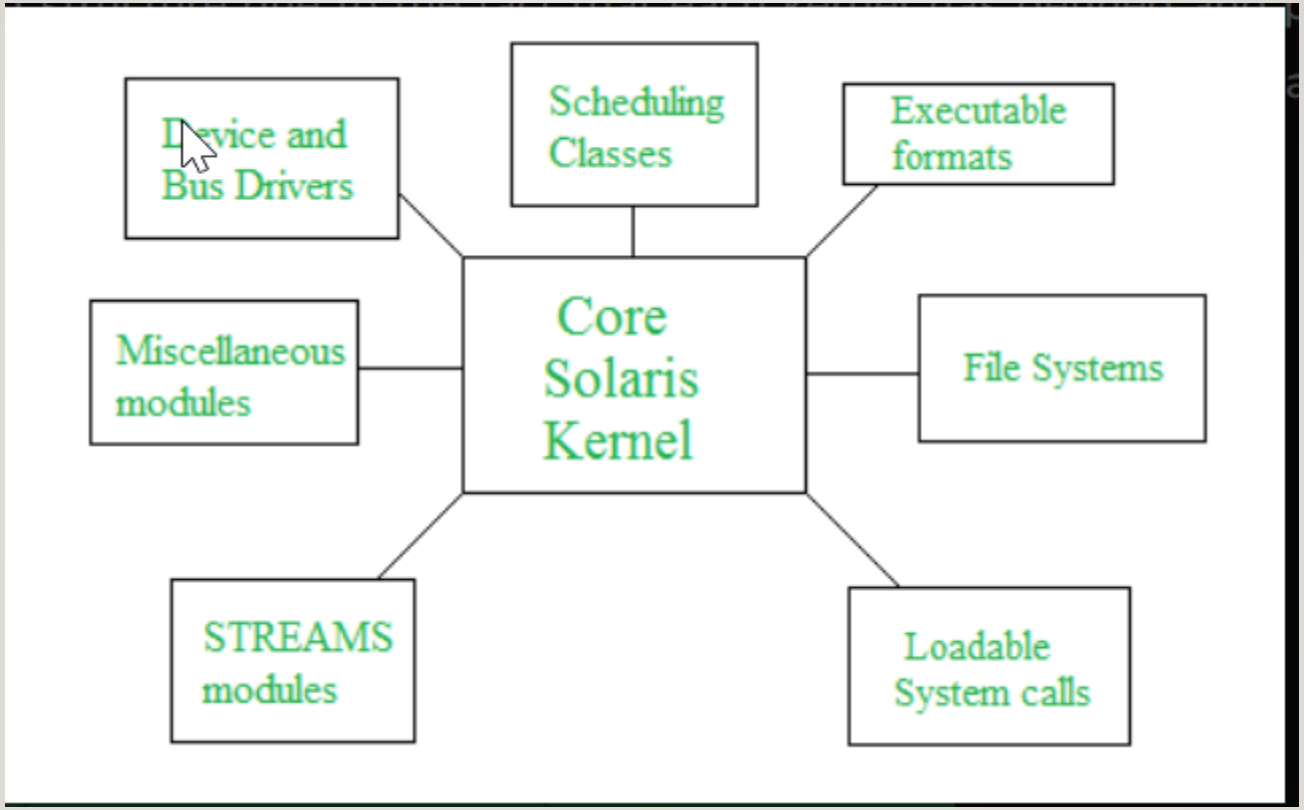
Layered structure

- An OS can be broken into pieces and retain much more control over the system.
- In this structure, the OS is broken into a number of layers (levels).
- The bottom layer (layer 0) is the hardware, and the topmost layer (layer N) is the user interface.
- These layers are so designed that each layer uses the functions of the lower-level layers.
- This simplifies the debugging process, if lower-level layers are debugged and an error occurs during debugging, then the error must be on that layer only, as the lower-level layers have already been debugged.
- The main disadvantage of this structure is that at each layer, the data needs to be modified and passed on which adds overhead to the system.


Layer N
User Interface
.....
Layer 1
Layer 0
Hardware

Modular structure

- It is considered as the best approach for an OS. It involves designing of a modular kernel.
- The kernel has only a set of core components and other services are added as dynamically loadable modules to the kernel either during runtime or boot time.
- It resembles layered structure due to the fact that each kernel has defined and protected interfaces, but it is more flexible than a layered structure as a module can call any other module.
- For example Solaris OS is organized as shown in the figure.


Device and
Bus Drivers
Scheduling
Classes
Executable
formats
Miscellaneous
modules
Core
Solaris
Kernel
File Systems
STREAMS
modules
Loadable
System calls

Virtual Machines(VMs)

- A virtual machine abstracts the hardware of our personal computer, including the CPU, disc drives, RAM, and NIC into a variety of different execution contexts giving us the impression that each execution environment is a different computer.
- An OS enables us to run multiple processes concurrently while making it appear as though each one is using a different processor and virtual memory by using CPU scheduling and virtual techniques.
- The fundamental issue with the virtual machine technique is disc systems. Let's say the physical machine only has three disc drives, but it needs to host seven virtual machines.

Thank You!